

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития
Кафедра инфокоммуникаций

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
дисциплины «Объектно-ориентированное программирование»
Вариант 2

Выполнил:
Безруков Даниил Андреевич
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: элементы объектно-ориентированного программирования в языке Python

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Порядок выполнения работы:

Ссылка на репозиторий: https://github.com/pokachtononame/oop_lab_1

1. Выполняем примеры

Пример: класс Rational для работы с рациональными дробями. Реализует операции арифметики (сложение, вычитание, умножение, деление) и операции сравнения (больше, меньше, равно)

```
class Rational:
    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError()

        self.__numerator = abs(a)
        self.__denominator = abs(b)

        self.__reduce()

    # Сокращение дроби
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)

        c = gcd(self.__numerator, self.__denominator)

        self.__numerator //= c
        self.__denominator //= c

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

# Прочитать значение дроби с клавиатуры. Дробь вводится как a/b.
```

```

def read(self, prompt=None):
    line = input() if prompt is None else input(prompt)
    parts = list(map(int, line.split("/", maxsplit=1)))

    if parts[1] == 0:
        raise ValueError()

    self.__numerator = abs(parts[0])
    self.__denominator = abs(parts[1])

    self.__reduce()

# Вывести дробь на экран
def display(self):
    print(f"{self.__numerator}/{self.__denominator}")

# Сложение обыкновенных дробей.
def add(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError()

# Вычитание обыкновенных дробей.
def sub(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator - self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator

        return Rational(a, b)
    else:
        raise ValueError()

# Умножение обыкновенных дробей.
def mul(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.numerator
        b = self.denominator * rhs.denominator

        return Rational(a, b)
    else:
        raise ValueError()

# Деление обыкновенных дробей.
def div(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator

        return Rational(a, b)
    else:
        raise ValueError()

# Отношение обыкновенных дробей.
def equals(self, rhs):
    if isinstance(rhs, Rational):
        return (self.numerator == rhs.numerator) and (self.denominator ==
rhs.denominator)
    else:
        return False

```

```

def greater(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator

        return v1 > v2
    else:
        return False

def less(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator
        return v1 < v2
    else:
        return False

```

Листинг 1. Класс Rational

```

if __name__ == "__main__":
    r1 = Rational()
    r1.read('Введите первую дробь: ')

    r2 = Rational()
    r2.read('Введите вторую дробь: ')

    print('Сумма: ')
    r3 = r2.add(r1)
    r3.display()

    print('Разность: ')
    r4 = r2.sub(r1)
    r4.display()

    print('Произведение: ')
    r5 = r2.mul(r1)
    r5.display()

    print('Частное: ')
    r6 = r2.div(r1)
    r6.display()

```

Листинг 2. Код примера

```

Введите первую дробь: 5/7
Введите вторую дробь: 7/9
Сумма:
94/63
Разность:
4/63
Произведение:
5/9
Частное:
49/45

```

Рисунок 1. Результат выполнения примера

2. Выполняем индивидуальные задания

Задание 1: реализовать класс с двумя полями, с именами `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Реализовать методы:

- 1) метод инициализации `__init__`; метод должен значения аргументов на корректность;
- 2) ввод с клавиатуры `read`;
- 3) вывод на экран `display`.

Реализовать внешнюю функцию с именем `make_pair()`. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу. Поле `first` — дробное число; поле `second` — дробное число, показатель степени. Реализовать метод `power()` — возведение числа `first` в степень `second`. Метод должен правильно работать при любых допустимых значениях `first` и `second`.

Решение:

```
class Pair:
    def __init__(self, a=0.0, b=0.0):
        if isinstance(a, (float, int)) and isinstance(b, (float, int)):
            self.first = float(a)
            self.second = float(b)
        else:
            raise ValueError('Некорректный тип аргументов')

    def read(self, prompt=None):
        if not isinstance(prompt, str):
            raise ValueError('Приглашение должно быть строкой')
        if prompt is None:
            text = input()
        else:
            text = input(prompt)
        parts = text.split()
        if len(parts) != 2:
            raise ValueError('Ожидается два значения')
        self.first = float(parts[0])
        self.second = float(parts[1])

    def display(self):
        print(f'First: {self.first}, Second: {self.second}')

    def power(self):
        return pow(self.first, self.second)

def make_pair(first, second):
```

```

try:
    return Pair(first, second)
except ValueError as error:
    print(f'Ошибка создания пары: {error}')
    return None

```

Листинг 3. Класс Pair

```

if __name__ == "__main__":
    p1 = Pair(30, 2.5)
    p2 = make_pair(20.5, 3)
    p3 = Pair()
    p3.read('Введите пару: ')
    p1.display()
    p2.display()
    p3.display()
    print(f'power() для первой переменной: {p1.power():.4f}')
    print(f'power() для второй переменной: {p2.power():.4f}')
    print(f'power() для третьей переменной: {p3.power():.4f}')
    p4 = Pair(0, 0)
    print(f'power() для нулевой степени: {p4.power():.4f}')

```

Листинг 4. Тестовый код

```

Введите пару: 5 3
First: 30.0, Second: 2.5
First: 20.5, Second: 3.0
First: 5.0, Second: 3.0
power() для первой переменной: 4929.5030
power() для второй переменной: 8615.1250
power() для третьей переменной: 125.0000
power() для нулевой степени: 1.0000

```

Рисунок 2. Результат выполнения тестового кода

Задание 2. Создать класс ModelWindow для работы с моделями экранных окон. В качестве полей задаются: заголовок окна, координаты левого верхнего угла, размер по горизонтали, размер по вертикали, цвет окна, состояние «видимое/невидимое», состояние «с рамкой/без рамки». Координаты и размеры указываются в целых числах. Реализовать операции: передвижение окна по горизонтали, по вертикали; изменение высоты и/или ширины окна изменение цвета; изменение состояния, опрос состояния. Операции передвижения и изменения размера должны осуществлять проверку на пересечение границ экрана. Функция вывода на экран должна индуцировать состояние полей объекта.

Решение:

```

class ModelWindow:
    screenX = 1600
    screenY = 900
    ALLOWED_COLORS = ['Красный', 'Зеленый', 'Синий', 'Белый', 'Черный']
    def __init__(self, title, color='Белый', startx=0, starty=0, width=100,
height=100, visible=True, borderless=False):
        if isinstance(title, str):
            self.title = title
        else:
            raise ValueError('Заголовок должен быть строкой')

        if isinstance(startx, int) and isinstance(starty, int):
            self.x = startx
            self.y = starty
        else:
            raise ValueError('Координаты должны быть целыми числами')

        if isinstance(width, int) and isinstance(height, int) and width > 0
and height > 0:
            self.width = width
            self.height = height
        else:
            raise ValueError('Размеры должны быть целыми положительными
числами')

        if color in self.ALLOWED_COLORS:
            self.color = color
        else:
            raise ValueError(f'Значение не в списке цветов')

        self.visible = bool(visible)
        self.borderless = bool(borderless)

    def change_color(self, new_color):
        if new_color not in self.ALLOWED_COLORS:
            raise ValueError('Значение не в списке цветов')
        self.color = new_color

    def display(self):
        print(f'\nСостояние окна {self.title}')
        print(f'Координаты (X, Y): {self.x}, {self.y}')
        print(f'Размеры (Ш x В): {self.width} x {self.height}')
        print(f'Цвет окна: {self.color}')
        print(f'Видимость: {'Видимое' if self.visible else 'Невидимое'})
        print(f'Рамка: {'Без рамки' if self.borderless else 'С рамкой'})

    def read(self):
        self.title = input('Введите заголовок окна: ')

        while True:
            try:
                x = int(input('Введите координату X: '))
                y = int(input('Введите координату Y: '))
                if x > self.screenX or y > self.screenY or x < 0 or y < 0:
                    print('Угол окна не должен выходить за пределы экрана')
                    continue
                self.x = x
                self.y = y
                break
            except ValueError:
                print('Координаты должны быть целыми числами. Попробуйте

```

```

снова.')

    while True:
        try:
            w = int(input('Введите ширину окна: '))
            h = int(input('Введите высоту окна: '))
            if w <= 0 or h <= 0:
                print('Ширина и высота должны быть больше нуля')
                continue
            if self.x + w > self.screenX or self.y + h > self.screenY:
                print('Окно не должно вылезать за рамки')
                continue
            self.width = w
            self.height = h
            break
        except ValueError:
            print('Размеры должны быть целыми числами. Попробуйте
снова.')

    while True:
        c = input(f'Введите цвет {self.ALLOWED_COLORS}: ').strip()
        if c in self.ALLOWED_COLORS:
            self.color = c
            break
        print('Такого цвета нет в списке. Попробуйте снова.')

    vis = input('Окно видимое? (да/нет): ').lower().strip()
    self.visible = (vis == 'да')

    bord = input('Окно без рамки? (да/нет): ').lower().strip()
    self.borderless = (bord == 'да')

    def shift(self, x, y):
        if isinstance(x, int) and isinstance(y, int):
            if self.x + x < 0 or self.x + x + self.width > self.screenX:
                print('Выход за границы экрана по X')
                return

            if self.y + y < 0 or self.y + y + self.height > self.screenY:
                print('Выход за границы экрана по Y')
                return

            self.x = self.x + x
            self.y = self.y + y
        else:
            raise ValueError('Смещения по координатам должны быть целыми
числами')

    def resize(self, width, height):
        if isinstance(width, int) and isinstance(height, int):

            if width <= 0 or height <= 0:
                print('Размеры должны быть больше нуля')
                return

            if self.x + width > self.screenX:
                print('Выход за границы экрана по X')
                return

            if self.y + height > self.screenY:
                print('Выход за границы экрана по Y')
                return

```



```

        self.width = width
        self.height = height
    else:
        raise ValueError('Размеры окон должны быть целыми числами')

    def set_visible(self, status):
        self.visible = bool(status)

    def set_borderless(self, status):
        self.borderless = bool(status)

    def is_visible(self):
        return self.visible

    def is_borderless(self):
        return self.borderless

```

Листинг 5. Код класса ModelWindow

```

if __name__ == "__main__":
    my_window = ModelWindow(title="Telegram", color="Синий", startx=100,
starty=100, width=400, height=300)
    my_window.display()

    my_window.shift(200, 50)
    my_window.shift(1700, 0)
    my_window.display()

    my_window.resize(800, 600)
    my_window.resize(800, 800)
    my_window.display()

    print(f"Сейчас окно видимое? {my_window.is_visible()}")
    my_window.set_visible(False)
    my_window.set_borderless(True)
    my_window.change_color("Красный")
    my_window.display()

    my_window.read()
    my_window.display()

```

Листинг 6. Тестовый код для класса ModelWindow

```
Введите заголовок окна: chrome
Введите координату X: 220
Введите координату Y: 250
Введите ширину окна: 1000
Введите высоту окна: 800
Окно не должно вылезать за рамки
Введите ширину окна: 1000
Введите высоту окна: 500
Введите цвет ['Красный', 'Зеленый', 'Синий', 'Белый', 'Черный']: Белый
Окно видимое? (да/нет): да
Окно без рамки? (да/нет): нет

Состояние окна chrome
Координаты (X, Y): 220, 250
Размеры (Ш x В): 1000 x 500
Цвет окна: Белый
Видимость: Видимое
Рамка: С рамкой
```

Рисунок 3. Отрывок результата выполнения кода выше

3. Отправляем коммиты в репозиторий
4. Оформляем отчёт в формате PDF и отправляем в папку doc репозитория

Ответы на контрольные вопросы:

1. Как осуществляется объявление класса в языке Python?

Для создания класса применяется ключевое слово `class`, после которого указывается имя класса с заглавной буквы (по правилам CamelCase), двоеточие и блок кода с отступами:

```
class MyClass:
    # Атрибуты (переменные) класса
    variable = 10

    # Методы класса
    def do_something(self):
```

pass

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса создаются непосредственно внутри тела класса (вне методов). Они являются общими для всех объектов этого класса и разделяют одно значение между ними (работают как глобальные константы или значения по умолчанию).

Атрибуты экземпляра объявляются внутри методов (обычно в `__init__`) через ссылку `self`. Они уникальны для каждого конкретного объекта и хранят его индивидуальное состояние (например, координаты конкретного окна).

3. Каково назначение методов класса?

Методы класса представляют собой функции, которые описывают поведение объектов. Они используются для взаимодействия с данными объекта: могут считывать его текущие свойства, изменять состояние полей (например, перемещать координаты или менять цвет) или выполнять связанные с этим объектом вычисления.

4. Для чего предназначен метод `__init__()` класса?

Метод `__init__` выполняет роль конструктора класса. Он вызывается автоматически в момент создания нового объекта. Его главная задача — инициализировать (задать) начальные значения для атрибутов экземпляра и выполнить базовые проверки данных при старте.

5. Каково назначение `self`?

Параметр `self` — это ссылка на текущий конкретный объект (экземпляр), для которого вызывается метод. Через `self` программа понимает, к атрибутам и методам какого именно объекта нужно обратиться изнутри кода класса. Он всегда идет первым аргументом в методах экземпляра.

6. Как добавить атрибуты в класс?

Чтобы добавить атрибут класса, его достаточно объявить прямой записью внутри тела класса.

Чтобы добавить атрибут экземпляра, его создают внутри методов (чаще всего в конструкторе `__init__`) путем присвоения значения через ссылку `self`, например: `self.name = value`.

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

В Python нет строгого разделения на приватные и публичные данные на уровне компилятора, доступ регулируется соглашением об именовании:

Публичные: пишутся без подчеркиваний (например, `title`, `shift`).

Защищенные: начинаются с одного подчеркивания (например, `_color`).

Приватные (скрытые на уровне класса): начинаются с двух подчеркиваний (например, `__status`). Python автоматически подменяет их имена, чтобы к ним нельзя было обратиться напрямую извне.

8. Каково назначение функции `isinstance`?

Встроенная функция `isinstance(object, classinfo)` используется для проверки типа данных. Она возвращает `True`, если проверяемый объект относится к указанному классу или к любому из его подклассов (наследников). Это позволяет безопасно проверять типы аргументов (например, перед выполнением арифметических действий или валидацией ввода).

Вывод: в ходе выполнения лабораторной работы были успешно освоены ключевые концепции объектно-ориентированного программирования и приобретены практические навыки по работе с классами и объектами при написании программ на языке Python версии 3.x. На примере класса рациональной дроби были детально изучены процессы объявления классов, инициализации уникальных атрибутов экземпляров с помощью конструктора, а также был разработан класс `ModelWindow` и методы для управления его состоянием.